

Lab Task 1: Hospital Patient Priority Queue (AVL Tree)

Hospitals need to manage patient admissions dynamically while ensuring that **critical patients (lower priority numbers)** are treated first.

Since new patients are constantly being added and discharged, the system must always stay efficient and balanced — ensuring fast insertion, lookup, and removal regardless of input order.

To achieve this, the hospital uses an **AVL Tree** as its internal data structure, where:

- Each node represents a patient.
- The tree is **ordered by patient priority** (lower number = higher priority).
- The tree automatically **rebalances itself** after every insertion and discharge to maintain $O(\log n)$ performance.

Data Model

Each AVL node represents one patient record.

```
struct Patient {  
    int priority;           // Smaller = more urgent  
    char name[50];         // Patient name  
    char condition[100];   // Description (e.g., "Fracture", "Heart  
Attack")  
};
```

Each AVL node will store:

```
struct Node {  
    Patient data;  
    Node* left;  
    Node* right;  
    int height;  
};
```

Note: Each node *wraps* a **Patient** and adds metadata needed for AVL balancing.

Functional Requirements

1. Insertion — `addPatient()`

- Inserts a new patient into the AVL Tree using `priority` as the key.
- Smaller numbers represent higher urgency.
- Reject duplicate priority numbers and return `false`.
- Automatically performs rebalancing if imbalance occurs.

2. Deletion — `removePatient()`

- Removes a patient with the given priority.
- Rebalances the tree after removal.
- If no such patient exists, return `false`.

3. Search — `findPatient()`

- Returns a pointer to the `Patient` struct if found.
- Returns `nullptr` if not found.

4. Highest and Lowest Priority Queries

- `getHighestPriorityPatient()` → returns the patient with the **lowest numeric priority value**.
- `getLowestPriorityPatient()` → returns the patient with the **highest numeric priority value**.
- If the tree is empty, return a `Patient` with `priority = -1`.

5. Range Query — `getPatientsInRange(low, high, outCount)`

- Returns all patients whose priority value lies between `low` and `high` (inclusive).
- Results are stored in a **dynamically allocated array** (`new Patient[count]`), ordered by increasing priority.
- Caller must free memory using `delete[]`.

6. Diagnostics

- `isBalanced()` — returns `true` if all balance factors $\in \{-1, 0, 1\}$.
- `getTotalPatients()` — total number of nodes.
- `getHeight()` — current tree height.

7. Display

- `displayInorder()` — prints all patients (priority, name, condition) in ascending order of priority.

Test Cases

```
#include "gtest/gtest.h"

#include "PatientAVL.h"

// 1. Add and Find Patient

TEST(PatientAVLTest, AddAndFindPatient) {

    PatientAVL avl;

    EXPECT_TRUE(avl.addPatient(5, "Alice", "Fracture"));

    EXPECT_TRUE(avl.addPatient(2, "Bob", "Heart Attack"));

    Patient* p = avl.findPatient(5);
```

```

    ASSERT_TRUE(p != nullptr);

    EXPECT_STREQ(p->name, "Alice");

    EXPECT_STREQ(p->condition, "Fracture");

    EXPECT_EQ(p->priority, 5);
}

// 2. Reject Duplicate Priorities
TEST(PatientAVLTest, RejectDuplicatePriorities) {
    PatientAVL avl;

    EXPECT_TRUE(avl.addPatient(10, "X", "Headache"));

    EXPECT_FALSE(avl.addPatient(10, "Y", "Fever"));

    EXPECT_EQ(avl.getTotalPatients(), 1);
}

// 3. Auto-Balancing After Multiple Inserts (LR Rotation)
TEST(PatientAVLTest, AutoBalancingAfterInsert) {
    PatientAVL avl;

    avl.addPatient(30, "A", "Cold");

    avl.addPatient(10, "B", "Flu");

    avl.addPatient(20, "C", "Asthma"); // triggers LR rotation

    EXPECT_TRUE(avl.isBalanced());
}

```

```
// 4. Delete Patient and Maintain Balance

TEST(PatientAVLTest, DeleteAndRebalance) {

    PatientAVL avl;

    avl.addPatient(10, "John", "Fever");

    avl.addPatient(20, "Mark", "Injury");

    avl.addPatient(30, "Sara", "Allergy");


    EXPECT_TRUE(avl.removePatient(10));

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_EQ(avl.getTotalPatients(), 2);

}


// 5. Delete Non-Existing Patient

TEST(PatientAVLTest, DeleteNonExistingPatient) {

    PatientAVL avl;

    avl.addPatient(1, "A", "Cold");

    EXPECT_FALSE(avl.removePatient(50)); // Not in tree

    EXPECT_TRUE(avl.isBalanced());

}


// 6. Get Highest and Lowest Priority Patients

TEST(PatientAVLTest, HighestLowestPriority) {

    PatientAVL avl;
```

```

    avl.addPatient(3, "P1", "Flu");

    avl.addPatient(1, "P2", "Heart Attack");

    avl.addPatient(5, "P3", "Cough");

    Patient high = avl.getHighestPriorityPatient();

    Patient low = avl.getLowestPriorityPatient();

    EXPECT_STREQ(high.name, "P2"); // smallest priority

    EXPECT_STREQ(low.name, "P3"); // largest priority
}

// 7. Range Query (getPatientsInRange)

TEST(PatientAVLTest, RangeQuery) {

    PatientAVL avl;

    avl.addPatient(1, "A", "X");

    avl.addPatient(2, "B", "Y");

    avl.addPatient(4, "C", "Z");

    avl.addPatient(6, "D", "W");

    int count = 0;

    Patient* result = avl.getPatientsInRange(2, 5, count);

    EXPECT_EQ(count, 2); // priorities 2,4

    EXPECT_EQ(result[0].priority, 2);

```

```
    EXPECT_EQ(result[1].priority, 4);

    delete[] result;
}

// 8. Find Non-Existing Patient Returns Null
TEST(PatientAVLTest, FindNonExisting) {

    PatientAVL avl;

    avl.addPatient(10, "Alex", "Cold");

    EXPECT_TRUE(avl.findPatient(99) == nullptr);
}

// 9. Delete All Patients
TEST(PatientAVLTest, DeleteAllPatients) {

    PatientAVL avl;

    avl.addPatient(1, "A", "Condition");
    avl.addPatient(2, "B", "Condition");
    avl.addPatient(3, "C", "Condition");

    avl.removePatient(1);
    avl.removePatient(2);
    avl.removePatient(3);

    EXPECT_EQ(avl.getTotalPatients(), 0);
}
```

```

    EXPECT_TRUE(avl.isBalanced());
}

// 10. Empty Tree Highest/Lowest Should Return Dummy
TEST(PatientAVLTest, EmptyTreeEdgeCase) {
    PatientAVL avl;

    Patient high = avl.getHighestPriorityPatient();

    Patient low = avl.getLowestPriorityPatient();

    EXPECT_EQ(high.priority, -1);

    EXPECT_EQ(low.priority, -1);
}

// 11. Sequential Insertions and Deletions Maintain Balance
TEST(PatientAVLTest, LargeSequence) {
    PatientAVL avl;

    for (int i = 1; i <= 20; ++i)
        avl.addPatient(i, "Temp", "Cond");

    for (int i = 2; i <= 20; i += 2)
        avl.removePatient(i);

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_EQ(avl.getTotalPatients(), 10);
}

```



```

// 12. Reinsert After Deletion Works Correctly

TEST(PatientAVLTest, ReinsertionAfterDelete) {

    PatientAVL avl;

    avl.addPatient(10, "A", "Injury");

    avl.removePatient(10);

    EXPECT_TRUE(avl.addPatient(10, "A", "Recovered"));

    Patient* p = avl.findPatient(10);

    ASSERT_TRUE(p != nullptr);

    EXPECT_STREQ(p->condition, "Recovered");

}

// 13. Tree Height After Multiple Inserts

TEST(PatientAVLTest, HeightAfterInsertions) {

    PatientAVL avl;

    for (int i = 1; i <= 15; ++i)

        avl.addPatient(i, "X", "Cond");

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_LT(avl.getHeight(), 8);

}

```

Lab Task 2 Self-Balancing Leaderboard System (AVL Tree)

You are building a **real-time leaderboard system** for a gaming platform.

Players earn and lose points continuously as they play, and the system must instantly reflect changes while keeping the leaderboard sorted by **score** in descending order.

Since players' scores change dynamically and thousands of updates occur per minute, using an ordinary list or array would become inefficient.

To handle this, the leaderboard uses a **self-balancing AVL Tree**, which automatically maintains order and guarantees **$O(\log n)$** time complexity for insertions, deletions, and updates.

Data Model

Each player record is represented by a struct:

```
struct Player {
    int playerId;    // Unique identifier
    char name[50];   // Player name
    int score;       // Player's current score
};
```

Each AVL node stores one **Player** record, plus height and balancing metadata:

```
struct Node {
    Player data;
    Node* left;
    Node* right;
    int height;
};
```

Note: The AVL Tree is ordered by score (descending).

If two players have the same score, use **playerID** to break ties (lower ID first).

Functional Requirements

1. *addPlayer(int id, const char name, int score)**

- Inserts a new player based on their **score** (higher score = higher rank).
- The **playerID** must be unique — reject duplicates and return **false**.

- The tree must automatically rebalance after insertion.
- Ordering Rule:
 - Higher **score** goes to **left** subtree.
 - Lower **score** goes to **right** subtree.
 - Equal score → compare **playerID**.

2. removePlayer(int id)

- Removes the player matching the given **playerID**.
- After removal, the tree must remain balanced.
- Returns **false** if the player is not found.

3. updateScore(int id, int newScore)

- Updates an existing player's score.
- Implemented as "remove old record → reinsert new record".
- Ensures tree remains AVL-balanced.
- Returns **false** if player not found.

4. findPlayer(int id)

- Searches the tree by **playerID** and returns a pointer to the **Player** struct.
- Returns **nullptr** if not found.

5. getTopPlayer()

- Returns the player with the **highest score**

6. getLowestPlayer()

- Returns the player with the **lowest score** (rightmost node).

7. getTopKPlayers(int k, int& outCount)

- Returns the **top K players** as a dynamically allocated array.
- Uses an in-order traversal (descending order).
- Sets `outCount = actual number of players returned`.
- Caller must `delete[]` the array.

8. getRank(int id)

- Returns the **rank** (1 = highest score).
- The rank is computed based on number of players with strictly higher scores.
- Implemented by traversing and counting during search.
- Returns `-1` if player not found.

9. Diagnostics

- `isBalanced()` → verifies AVL property.

- `getHeight()` → returns tree height.
- `getTotalPlayers()` → total number of nodes.

10. Display

- `displayDescending()` → prints all players (name, score) in descending score order.

Test Cases

```
#include "gtest/gtest.h"
#include "LeaderboardAVL.h"

// 1. Add and Find Player
TEST(LeaderboardAVLTest, AddAndFindPlayer) {
    LeaderboardAVL avl;
    EXPECT_TRUE(avl.addPlayer(101, "Alice", 250));
    EXPECT_TRUE(avl.addPlayer(102, "Bob", 400));

    Player* p = avl.findPlayer(102);
    ASSERT_TRUE(p != nullptr);
    EXPECT_STREQ(p->name, "Bob");
    EXPECT_EQ(p->score, 400);
}

// 2. Reject Duplicate Player IDs
TEST(LeaderboardAVLTest, RejectDuplicateIDs) {
    LeaderboardAVL avl;
    EXPECT_TRUE(avl.addPlayer(1, "Player1", 100));
    EXPECT_FALSE(avl.addPlayer(1, "Player2", 200));
    EXPECT_EQ(avl.getTotalPlayers(), 1);
}

// 3. Auto-Balancing After Insertions
TEST(LeaderboardAVLTest, BalancingAfterInserts) {
```

```

LeaderboardAVL avl;
avl.addPlayer(1, "A", 300);
avl.addPlayer(2, "B", 500);
avl.addPlayer(3, "C", 400); // triggers rotation
EXPECT_TRUE(avl.isBalanced());
}

// 4. Update Score Changes Rank
TEST(LeaderboardAVLTest, UpdateScoreAffectsRank) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 300);
    avl.addPlayer(3, "C", 200);

    int rankBefore = avl.getRank(1); // A = lowest
    avl.updateScore(1, 350);          // A moves up
    int rankAfter = avl.getRank(1);

    EXPECT_GT(rankBefore, rankAfter); // rank improves
    EXPECT_TRUE(avl.isBalanced());
}

// 5. Remove Player and Maintain Balance
TEST(LeaderboardAVLTest, DeleteAndRebalance) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "X", 100);
    avl.addPlayer(2, "Y", 200);
    avl.addPlayer(3, "Z", 300);

    EXPECT_TRUE(avl.removePlayer(2));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_EQ(avl.getTotalPlayers(), 2);
}

// 6. Highest and Lowest Player
TEST(LeaderboardAVLTest, TopAndLowestPlayers) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 500);
    avl.addPlayer(3, "C", 300);

```

```

    Player top = avl.getTopPlayer();
    Player low = avl.getLowestPlayer();

    EXPECT_STREQ(top.name, "B");
    EXPECT_STREQ(low.name, "A");
}

// 7. Top K Players Retrieval
TEST(LeaderboardAVLTest, GetTopKPlayers) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 500);
    avl.addPlayer(3, "C", 300);
    avl.addPlayer(4, "D", 400);

    int count = 0;
    Player* top = avl.getTopKPlayers(3, count);
    EXPECT_EQ(count, 3);
    EXPECT_STREQ(top[0].name, "B"); // Highest score first
    EXPECT_STREQ(top[1].name, "D");
    EXPECT_STREQ(top[2].name, "C");
    delete[] top;
}

// 8. Get Rank Functionality
TEST(LeaderboardAVLTest, RankCalculation) {
    LeaderboardAVL avl;
    avl.addPlayer(1, "A", 100);
    avl.addPlayer(2, "B", 400);
    avl.addPlayer(3, "C", 300);

    EXPECT_EQ(avl.getRank(2), 1); // B highest
    EXPECT_EQ(avl.getRank(3), 2); // C second
    EXPECT_EQ(avl.getRank(1), 3); // A lowest
}

// 9. Delete Non-Existent Player
TEST(LeaderboardAVLTest, DeleteNonExisting) {
    LeaderboardAVL avl;

```

```
    avl.addPlayer(1, "A", 100);
    EXPECT_FALSE(avl.removePlayer(999));
    EXPECT_EQ(avl.getTotalPlayers(), 1);
}

// 10. Large Sequence Stress Test
TEST(LeaderboardAVLTest, LargeInsertDeleteSequence) {
    LeaderboardAVL avl;
    const int N = 50;
    for (int i = 1; i <= N; ++i)
        avl.addPlayer(i, "P", i * 10);
    for (int i = 1; i <= N; i += 3)
        avl.removePlayer(i);
    EXPECT_TRUE(avl.isBalanced());
}
```